Winter 390 Presentation 2: Team 5

Fauzan Aryaputra, Paris Bozzuti, Ronaldo Tineo, Emma Smith, Aidan Balagtas

January 2026

Splitting Benign Cases With >5 Slices

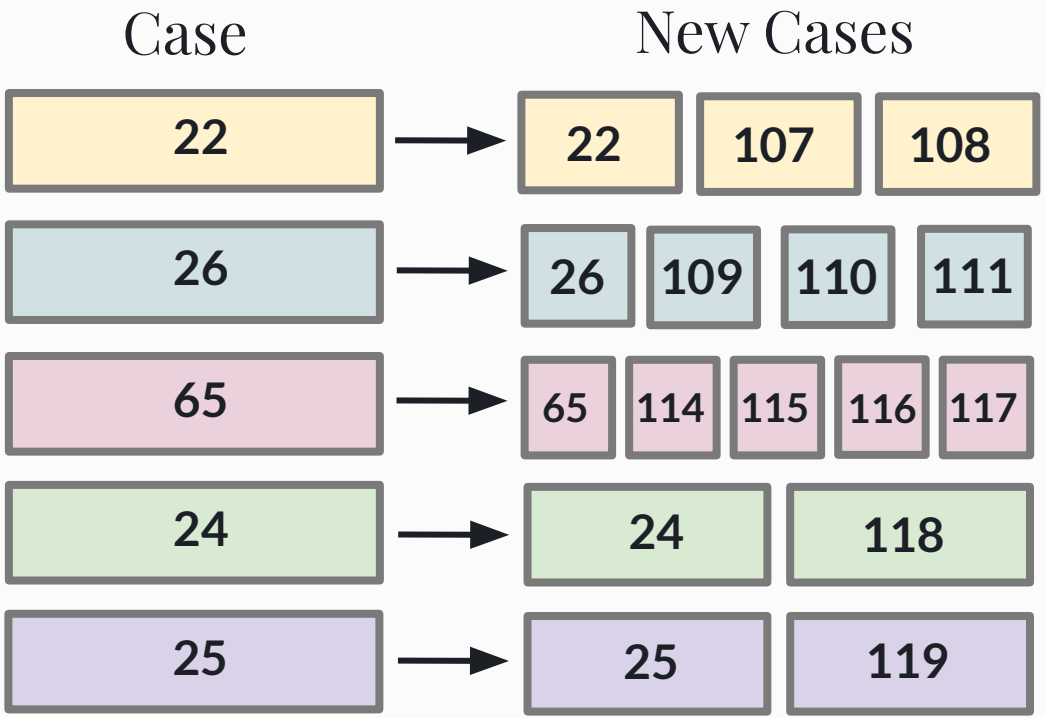
Aidan

Splitting Benign Cases

Goal: Make new benign cases because model only uses up to 5 slices per case

Approach: Use script to change patch names for each case. Update case_grade_match.csv to reflect new benign cases.

Benign cases with extra slices:
22, 24, 25, 26, 27, 65, 82



Splitting Benign Cases Cont.

Additional Notes:

- Kept track of number of patches per slice to roughly balance patch number between cases

- Maintained at least one sox10 slice per case

- Case 27 is missing patches in match 2 (no h&e/sox10 patches). Slices are in OneDrive, so revisit output of initial patching?

- Case 85 requires splitting patches between slices to accommodate lack of sox10 slices (can finish this in future)

Splitting Benign Cases Cont.

Case 22

Original Contains:

2 matches

Unmatched: 11 h&e, 1 melan, 1 sox10

Case 107:

Match 1, unmatched H&E slices 2/3/5

Case 108:

Match 2, 4 unmatched H&E slices 6/9/10/11

Case 22:

unmatched H&E slices 1/4/7/8, unmatched melan and sox10 slices 1

Case 24

Original Contains:

2 matches

Unmatched: 8 h&e, 2 melan, 0 sox10

Case 118:

Match 1, unmatched H&E slices 1/2/3/4, unmatched melan slices 9

Case 24:

Match 2, unmatched H&E slices 5/6/7/8, unmatched melan slices 10

Splitting Benign Cases Cont.

Case 25

Original Contains:

2 matches

Unmatched: 4 h&e, 0 melan, 0 sox10

Case 119:

Match 1, unmatched H&E slices 1/4

Case 25:

Match 2, unmatched H&E slices 2/3

Case 26

Original Contains:

3 matches

Unmatched: 14 h&e, 5 melan, 1 sox10

Case 109/110/111:

Match **1/2/3**, unmatched H&E slices **1-3/4-6/7-9**, unmatched melan slice **1/2/4**

Case 26:

unmatched h&e slices 10-14, unmatched melan slices 3/5, unmatched sox10 slice 1

Splitting Benign Cases Cont.

Case 65

Original Contains:

10 matches

Case 114: match 3/4

Case 115: match 5/6

Case 116: match 7/8

Case 117: match 9/10

Case 65: match 1/2

Confirmed all matches have all three slices.

Optuna: Hyperparameter Tuning

Paris

optuna_tuning_2.py

This script runs mini-trainings to find which hyperparameters work best, which can then be used for main.py final training.

How the script works:
Uses the same data loading, model architecture as the main.py training.

OptunaTrainer is a simplified version of **MILTrainer** that trains faster and uses trial-specific hyperparameters.

Optuna uses a study, which is an optimization based on an objective function, and a trial, which is a single execution of the function. The goal of the study is to test multiple trials to find the optimal set of hyperparameter values.

The objective function samples hyperparameters from the search space, trains/evaluates the model and returns a scalar score. It uses Bayesian optimization, so it uses past results to determine promising regions for hyperparameters. Each pitch for the next hyperparameters are based on the results of the previous trial.

optuna_tuning_2.py

This script runs mini-trainings to find which hyperparameters work best, which can then be used for main.py final training.

Tunes:

1. **Learning Parameters:**
    a. learning_rate
    b. weight_decay
    c. Dropout

Outputs:

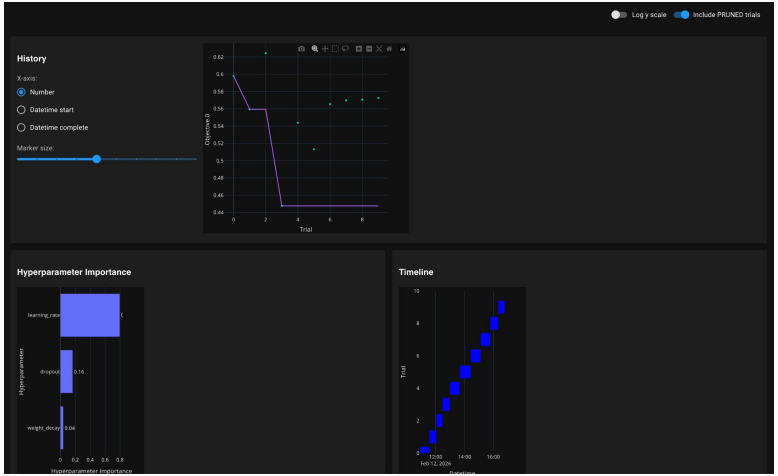
Best_hyperparameters.json: the best combination
All_trials.csv: results from all n trials
Best_config.py: code for config.py
Study_statistics.json: summary metadata
optuna.db: full database for dashboard functionality

optuna_tuning_2.py

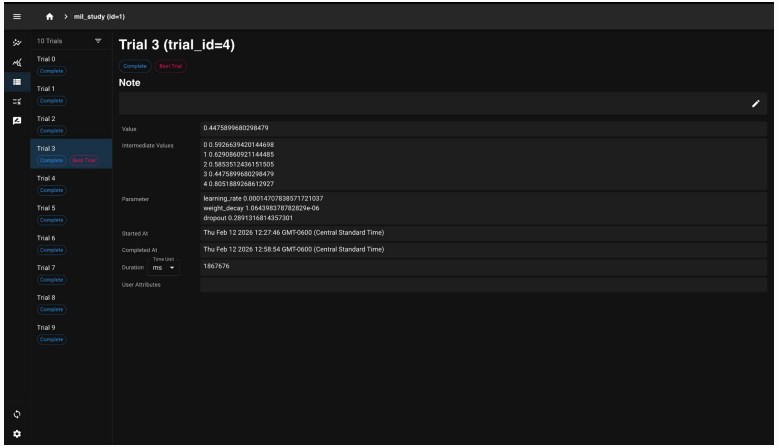
```python
def objective(trial, train_data, val_data, args):
    """
    Optuna objective function
    Suggests hyperparameters and returns validation loss
    """
    # Suggest hyperparameters (only learning parameters)
    trial_params = {
        'learning_rate': trial.suggest_float('learning_rate', 1e-5, 1e-3, log=True),
        'weight_decay': trial.suggest_float('weight_decay', 1e-6, 1e-3, log=True),
        'dropout': trial.suggest_float('dropout', 0.1, 0.5),

        # Fixed values
        'class_weights': [1.0, 1.0],
        'use_scheduler': False,
    }

    # Create model with fixed architecture from config
    model = create_model(
        num_classes=MODEL_CONFIG['num_classes'],
        embed_dim=MODEL_CONFIG['embed_dim'],
        dropout=trial_params['dropout']
    )
```

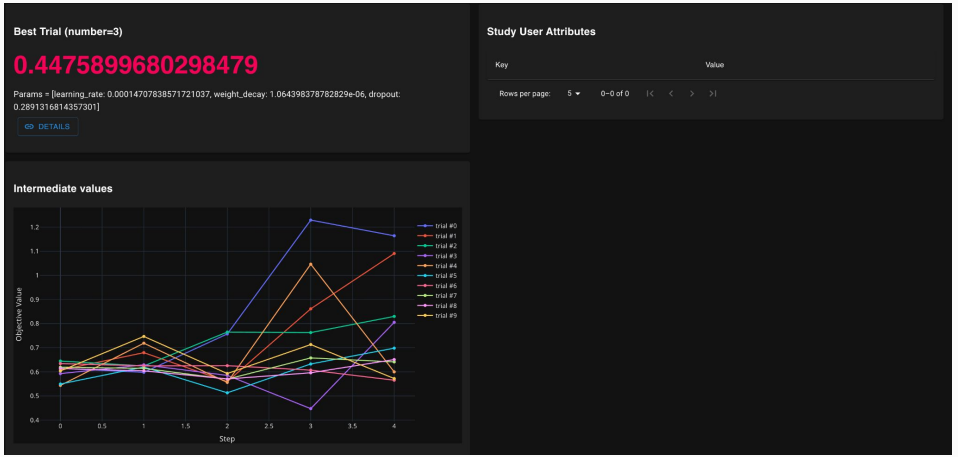
```python
def train_with_pruning(self, train_loader, val_loader, n_epochs):
    """Training loop with Optuna pruning"""
    for epoch in tqdm(range(n_epochs), desc="Epochs", leave=False):
        train_loss = self.train_epoch(train_loader)
        val_loss, val_acc = self.validate(val_loader)

        # Update learning rate
        if self.scheduler:
            self.scheduler.step(val_loss)

        # Track best validation loss
        if val_loss < self.best_val_loss:
            self.best_val_loss = val_loss

        # Report to Optuna and handle pruning
        if self.trial is not None:
            self.trial.report(val_loss, epoch)

            # Check if trial should be pruned
            if self.trial.should_prune():
                raise optuna.TrialPruned()

    return self.best_val_loss, val_acc
```

Dashboard Functionality

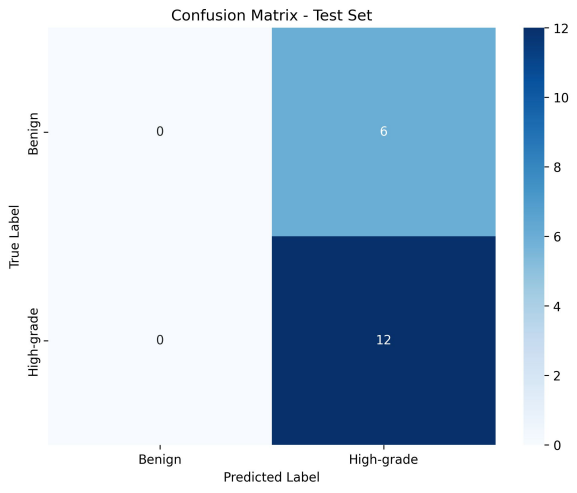


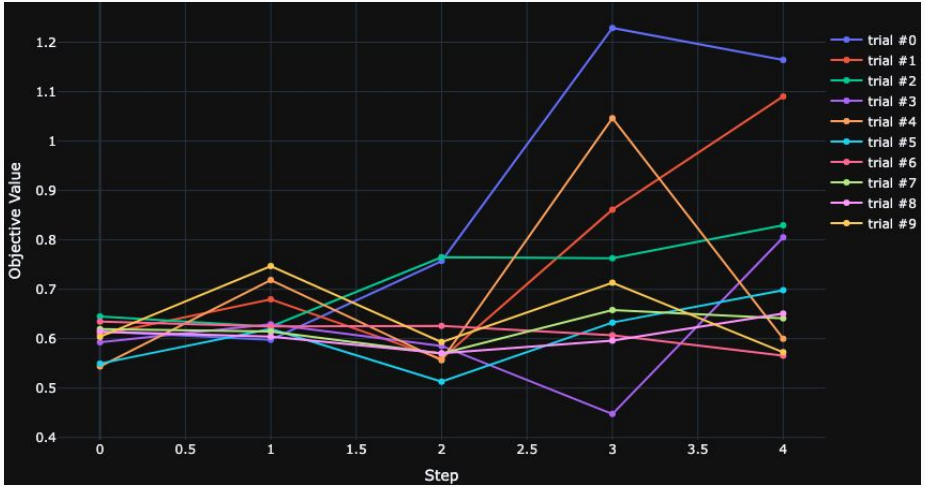
Every optuna tuning run can access a dashboard of the results as the study updates even over Quest. It creates a SQL database of all of the results, for each epoch within each trial. This allows you to manage the progress over the entire run of the tuning.





Results





The model predicted everything as high-grade, so clearly there is a problem. It is likely from not tuning the class weights, which is something we can try in a future iteration, now that we know how to use Optuna and how to train it in just a few hours.

Implementation: Entropy Regularization

Fauzan, Ronaldo

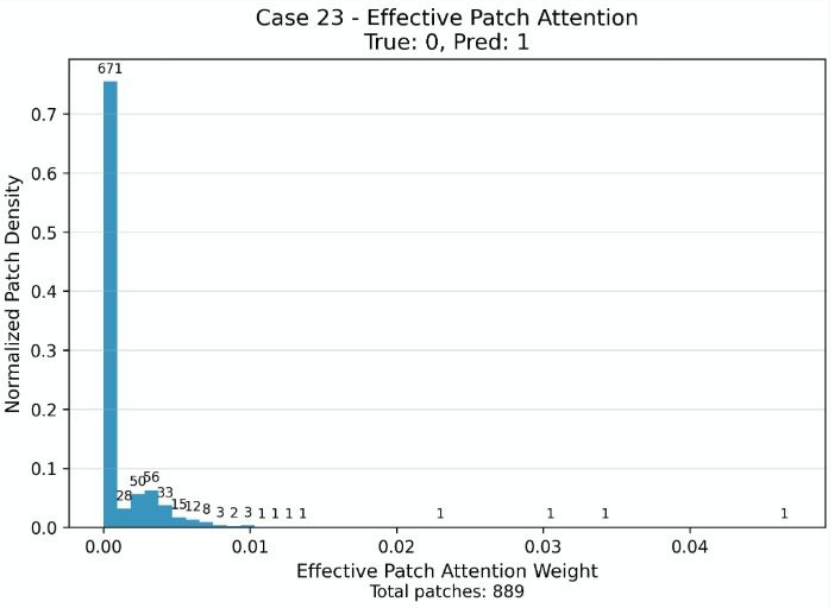
Motivation

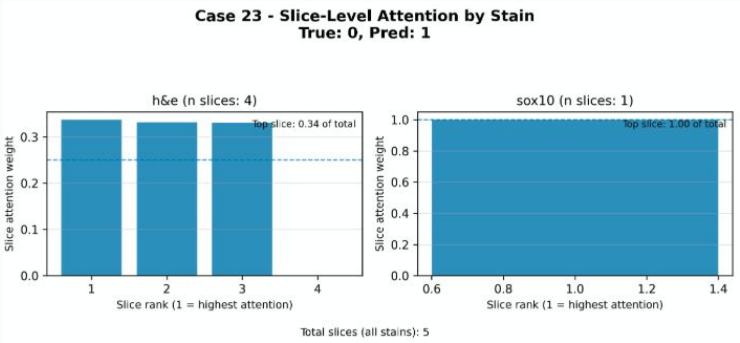
Provide theoretical downside protection:

- Entropy regularization of attention scores would help the false positive case, where benign patches are being predicted to be high grade
- Current model performance on the test set has:
  - Very peaky patch attention distributions for correctly predicted true cases
  - Slightly peaky patch attention distributions for benign cases
  - For all true_label = 1 cases, the model is very confident that it is positive (predicted probability is very high)
  - For all true_label = 0 cases, the model is less confident that it is negative
- If we were to regularize the distribution of attention scores, we would be able to get the model to “care less” about individual patches
  - Argument to be made that this peakiness is what is making the model predict true positive cases so confidently
  - I argue that this implies that we have a lot of rope, regularizing peaky patch attention scores will lead to better chances that no one patch completely dominates a whole case, which seems to be happening in these false positive cases

Motivation

Peaky attention scores:





Too confident that a benign prediction was positive!

Implementation

Changed models.py:

- __init__:
  - Added entropy regularization coefficient (self.lambda_entropy)
    - Pulled from TRAINING_CONFIG
    - Enables tuning regularization strength independently of other regularization methods
  - Added get_entropy()
    - Calculates entropy of a given instance ↠ w * log(w)
- train_epoch:
  - Modified forward call to optionally return entropy
    - outputs, entropy = self.model(..., return_entropy=True)
  - Added entropy regularization to loss
    - loss = loss - λ_entropy * entropy
    - Encourages smoother attention distributions
- Across the code:
  - Ensured that entropy was tracked throughout training and passed up the hierarchy in process_single_stain()
  - Added entropy to the training config

Implementation (cont.)

Changed trainer.py:

__init__:

```
self.lambda_entropy = TRAINING_CONFIG.get('lambda_entropy', 0.0)
```

train_epoch:

```
if self.lambda_entropy > 0:
    outputs, entropy = self.model(
        stain_slices,
        return_entropy=True
    )
else:
    outputs = self.model(stain_slices)
    entropy = None
```

```
if entropy is not None:
    loss = loss + self.lambda_entropy * entropy

# Backward pass
self.optimizer.zero_grad()
loss.backward()
self.optimizer.step()
```

Preliminary Results

- Ran into similar issues that Paris did - must be basing our iterations on a faulty version of the model that likes to predict the test case all high-grade

- We have the entropy model implemented, we just need to fix the rest of the model and rerun to visualize the new attention distributions

Theory: Optimal Dropout Usage

Fauzan

Current Model Regularization

After implementing entropy regularization, we currently have three main forms of regularization across the model:

1) Dropout
    a) Patch projector
    b) Attention MLP
    c) Final classifier
2) Weight decay (L2 reg)
    a) Global
3) Entropy regularization
    a) Attention MLP

Full Stylized Loss Function:

$$Loss = CrossEntropy + L2 + Entropy$$

Where is Dropout Implemented?

1) Patch Projector
   a) Adds noise to the instance embeddings

2) Attention MLP
   a) Adds noise to attention logits
   b) Makes attention distribution “jittery” batch to batch
   c) Just implemented entropy regularization

3) Final Classifier
   a) Regularizes the final representation

Agree with the intuition that this is too many places to implement dropout, especially for a small training dataset. I argue that we should remove dropout from the attention MLP.

Why Could Attention Dropout Be Problematic?

- Attention weights are:

$$w_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

  - Where logits z come from the attention MLP

- Where the derivative with respect to attention weights w is:

$$\frac{\partial w_i}{\partial z_i} = w_i(1 - w_i) \qquad \frac{\partial w_i}{\partial z_j} = -w_i w_j$$

- Suppose dropout injects noise:

$$z_k = z_k + \epsilon$$

Why Could Attention Dropout Be Problematic? (cont.)

- This means that variance in z propagates nonlinearly:
  - Using the noise from the previous slide, the change in attention weight is approximately:

$$\Delta w_i \approx \frac{\partial w_i}{\partial z_k} \epsilon$$

  - Small noise in logits can cause disproportionately larger changes in attention weights, especially when distributions are peaky

- Because the derivative of weights wrt logits are not constant, and coupled with the magnitude of the weights themselves:

Softmax converts additive noise in logits into multiplicative shifts in the probability!
Further, this effect is passed through all three layers, meaning that this attention weight propagates through nonlinearly

Effect of Hierarchical Architecture

- Mathematically, the final representation is some function with the structure:

$$Case = \sum_s w_s^{stain} \left(\sum_l w_{s,l}^{slide} \left(\sum_p w_{s,l,p}^{patch} x_{s,l,p} \right) \right)$$

  - So the final output is proportional to: $w^{stain} \times w^{slide} \times w^{patch}$

- Through three levels, after dropout, the final patch influence looks something like:

$$(w + \Delta w)^{stain} \times (w + \Delta w)^{slide} \times (w + \Delta w)^{patch}$$

- This means that the variance of the model increases like:

$$Var(product) \approx Var_1 + Var_2 + Var_3 + CrossTerms$$

  - Stochasticity increases nonlinearly with depth!

Effect of Hierarchical Architecture (cont.)

- Effect 1: Variance Amplification
  - Each level introduces logit noise
  - At three levels, noise is multiplicatively compounded and gradient variance increases

- Effect 2: Sharpening Existing Bias
  - As demonstrated in the previous slides, dropout can amplify the peakiness of the attention distribution of a given bag
  - We don’t mind the case where it shifts it toward making it less peaky, but it is still a stochastic process and can also increase the peakiness

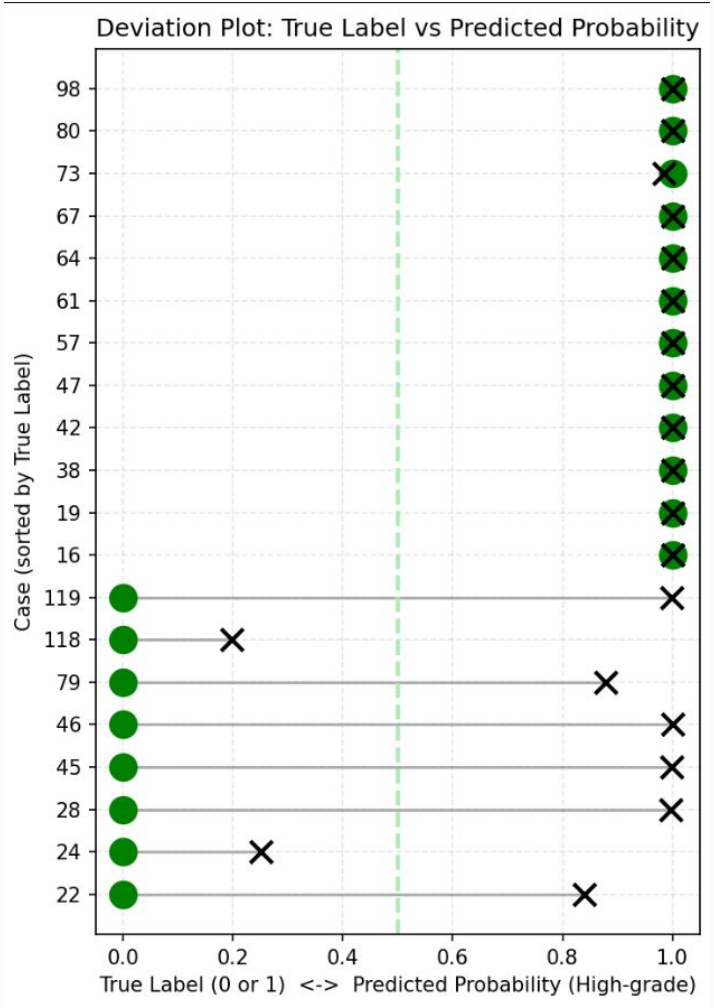
- Minor Effect 3: Gradient Attenuation For Low Weight Bag Instances

Recommendation

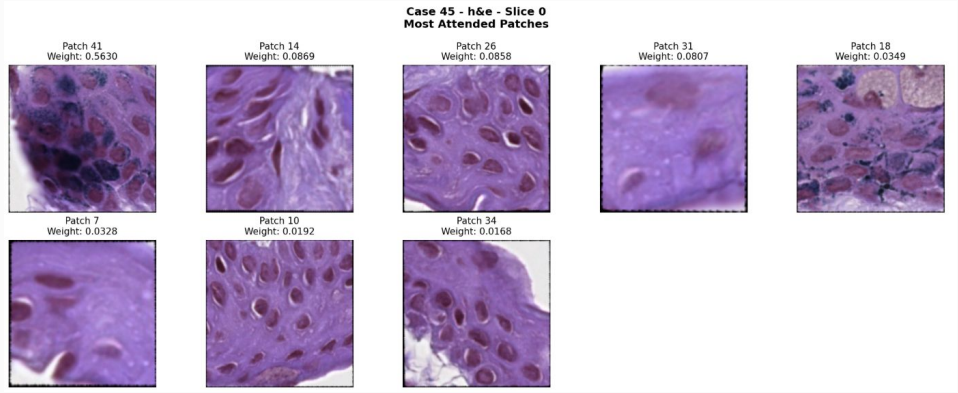
- Given that we are implementing attention weight entropy regularization,
- I recommend that we remove/significantly reduce dropout in the attention weight MLPs

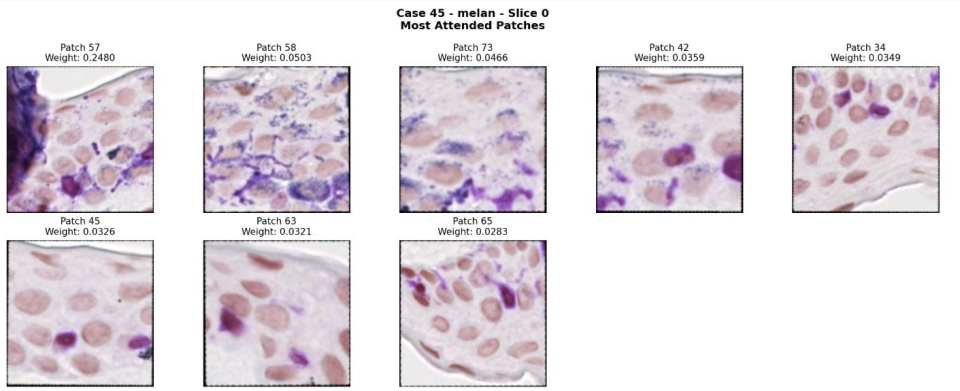
Analysis

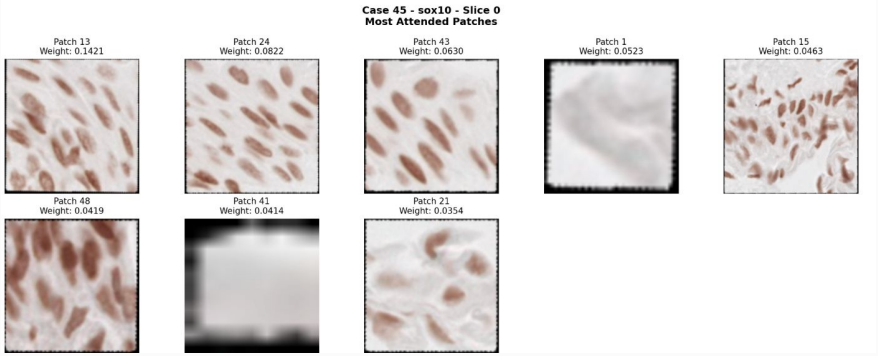
Emma, Ronaldo


Deviation Plot: True Label vs Predicted Probability
Case (sorted by True Label)
98
80
73
67
64
61
57
47
42
38
19
16
119
118
79
46
45
28
24
22
0.0
0.2
0.4
0.6
0.8
1.0
True Label (0 or 1) <-> Predicted Probability (High-grade)

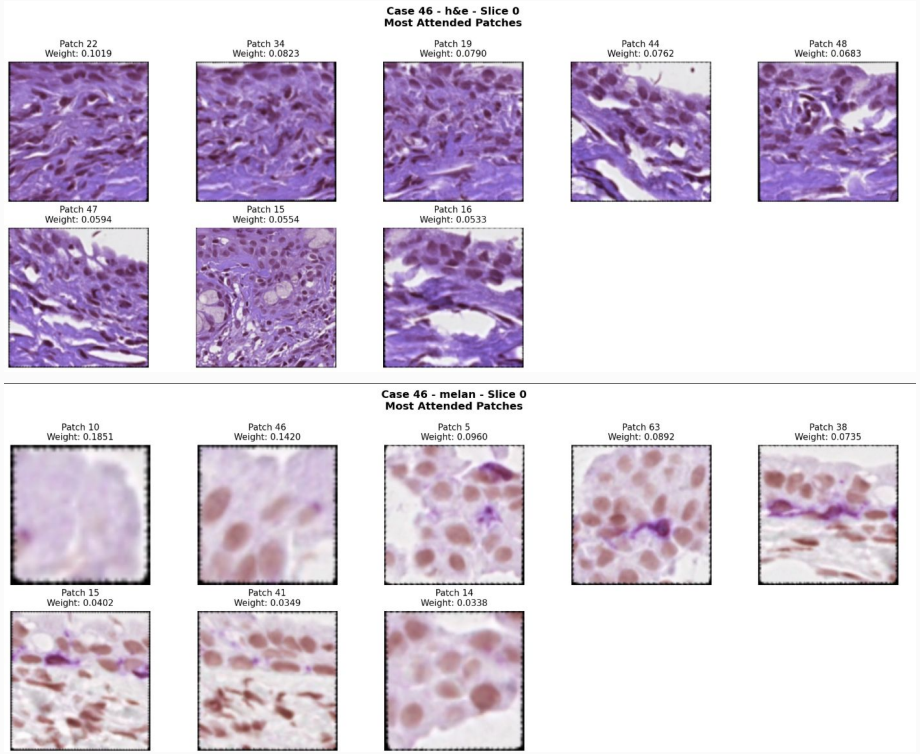
Incorrectly Classified High-Grade

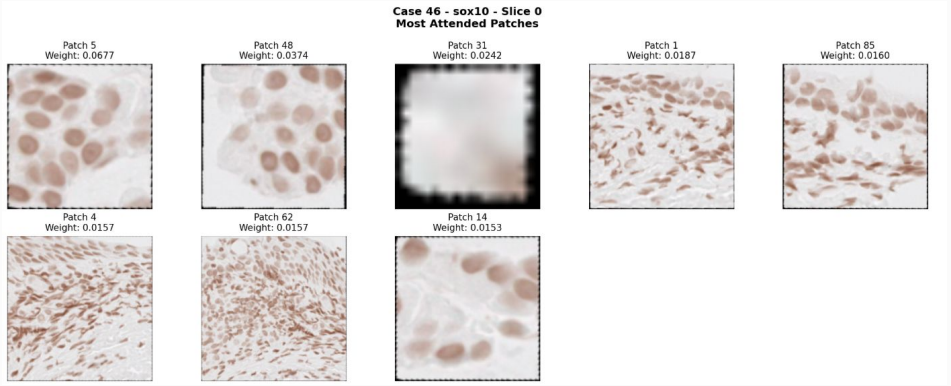




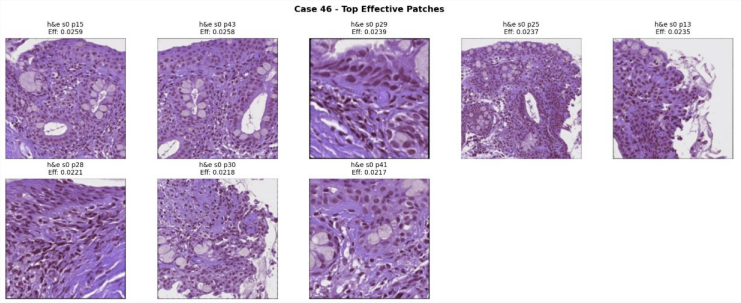


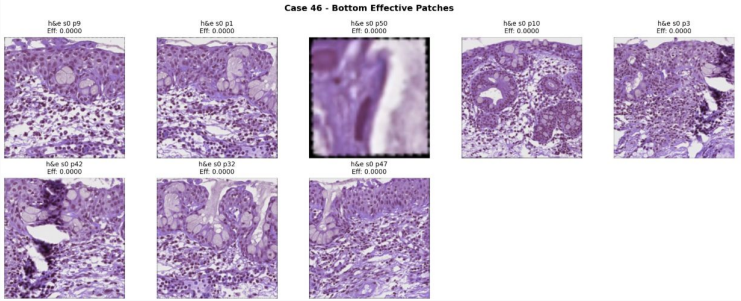
Incorrectly Classified High-Grade

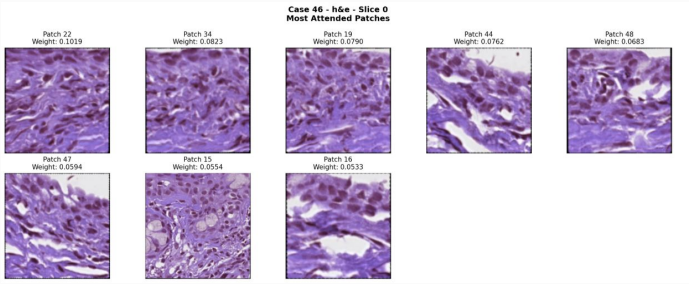


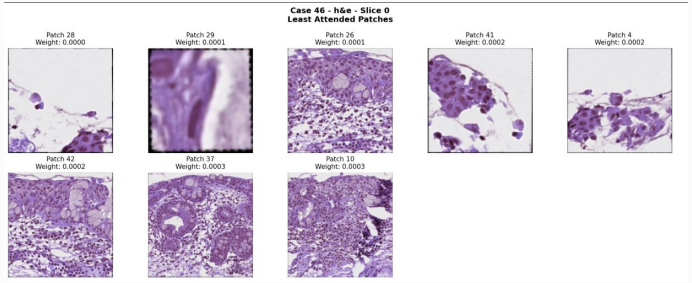


Incorrectly Classified High-Grade

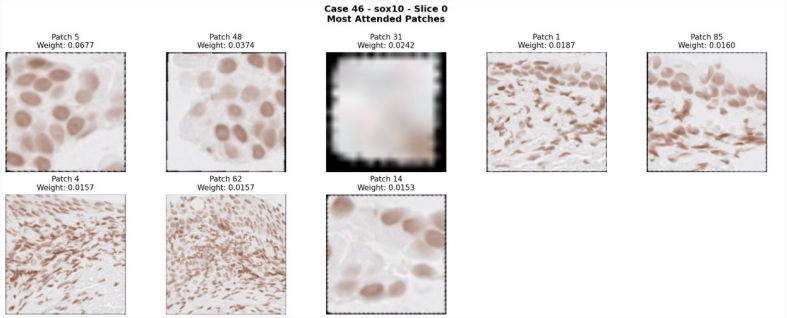


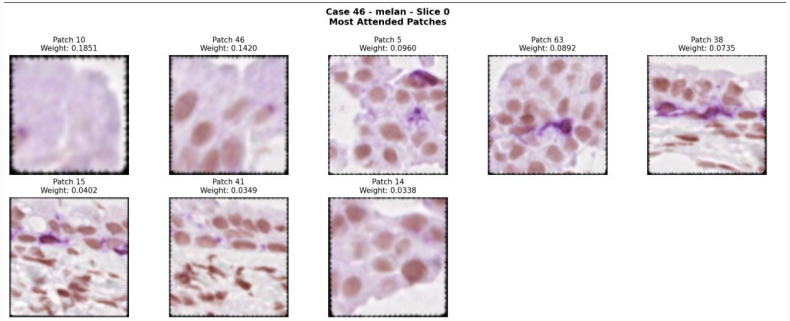


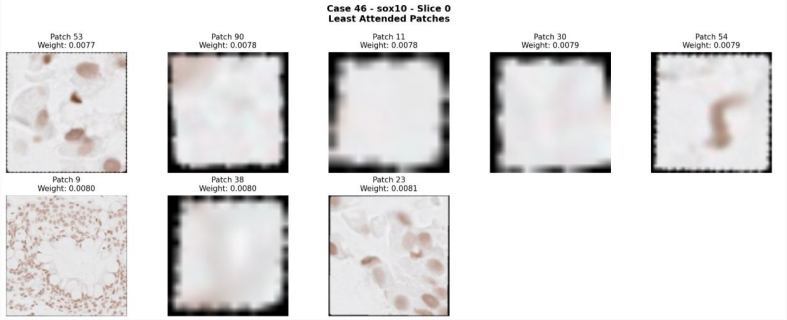


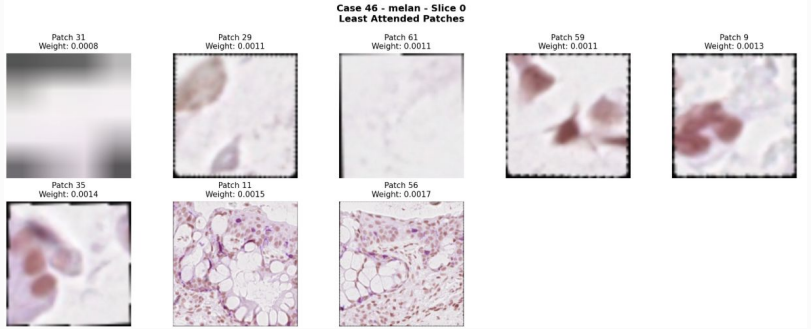


Case 46 cont.

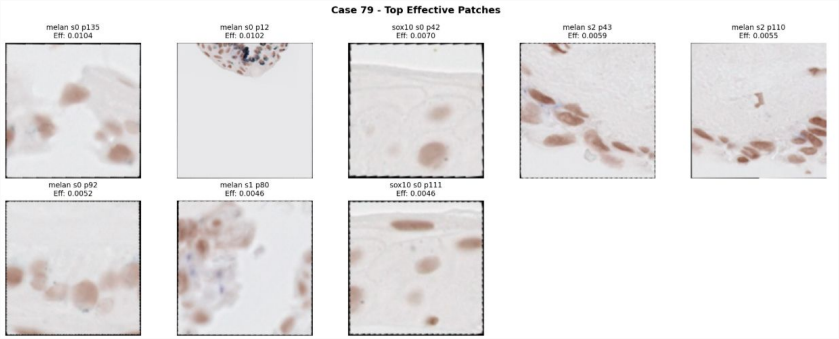

Case 46 - sox10 - Slice 0
Most Attended Patches
Patch 5
Weight: 0.0677
Patch 48
Weight: 0.0374
Patch 31
Weight: 0.0242
Patch 1
Weight: 0.0187
Patch 85
Weight: 0.0160
Patch 4
Weight: 0.0157
Patch 62
Weight: 0.0157
Patch 14
Weight: 0.0153

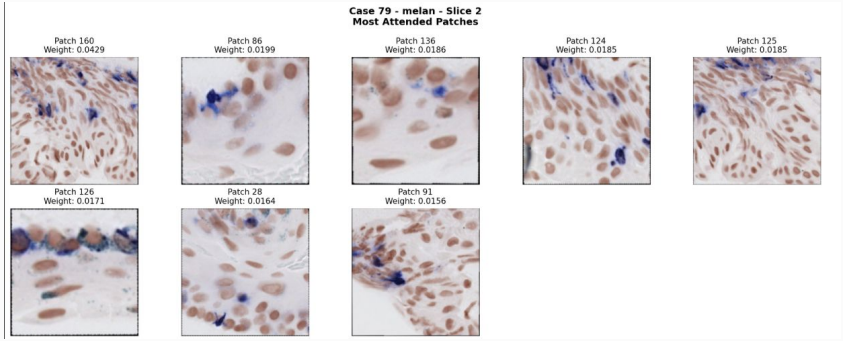

Case 46 - melan - Slice 0
Most Attended Patches
Patch 10
Weight: 0.1851
Patch 46
Weight: 0.1420
Patch 5
Weight: 0.0960
Patch 63
Weight: 0.0892
Patch 38
Weight: 0.0735
Patch 15
Weight: 0.0402
Patch 41
Weight: 0.0349
Patch 14
Weight: 0.0338

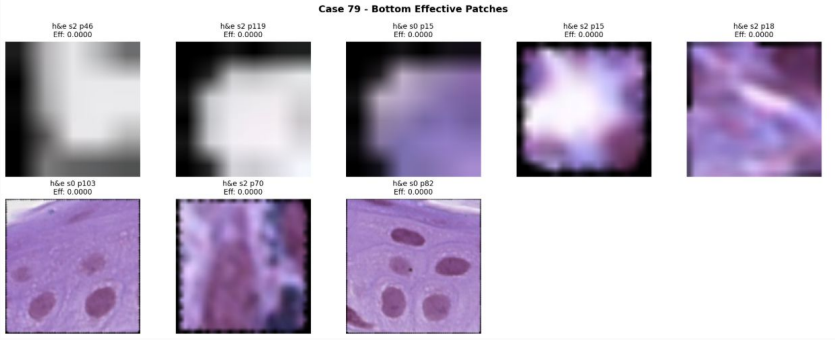

Case 46 - sox10 - Slice 0
Least Attended Patches
Patch 53
Weight: 0.0077
Patch 90
Weight: 0.0078
Patch 11
Weight: 0.0078
Patch 30
Weight: 0.0079
Patch 54
Weight: 0.0079
Patch 9
Weight: 0.0080
Patch 38
Weight: 0.0080
Patch 23
Weight: 0.0081

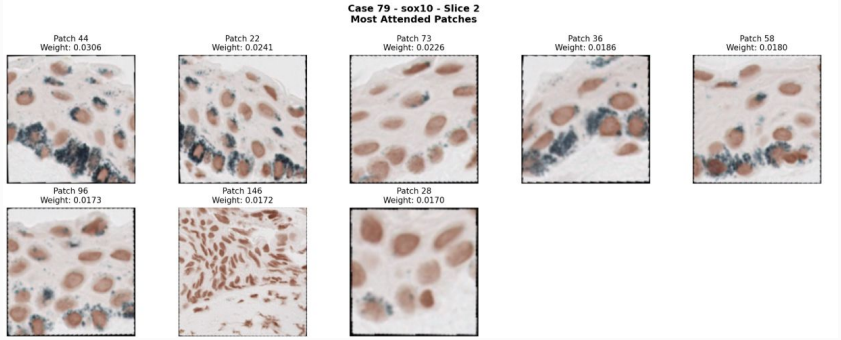

Case 46 - melan - Slice 0
Least Attended Patches
Patch 31
Weight: 0.0008
Patch 29
Weight: 0.0011
Patch 61
Weight: 0.0011
Patch 59
Weight: 0.0011
Patch 9
Weight: 0.0013
Patch 35
Weight: 0.0014
Patch 11
Weight: 0.0015
Patch 56
Weight: 0.0017

Incorrectly Classified High-Grade

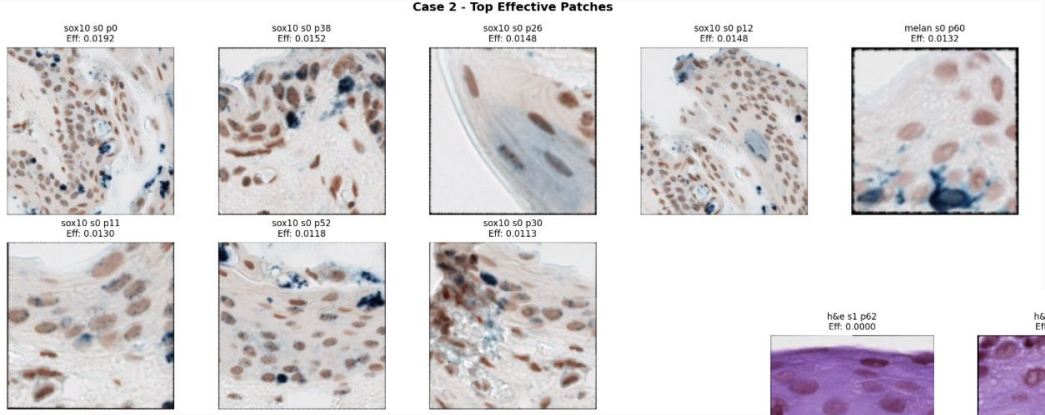


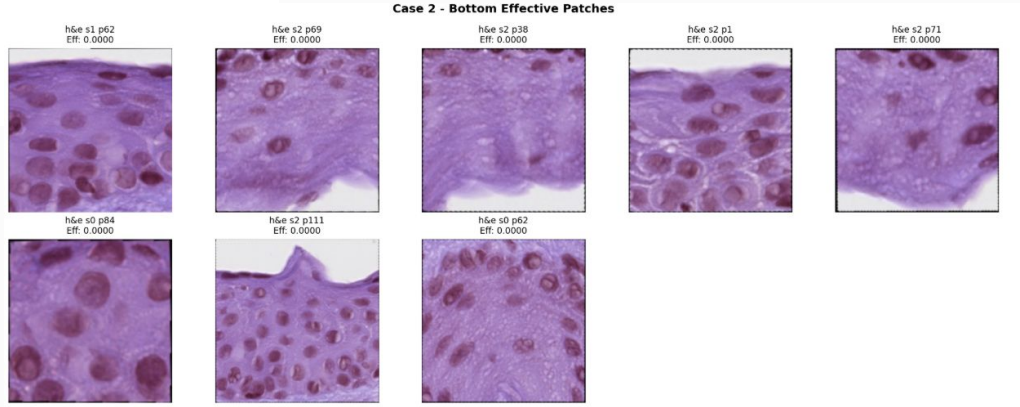




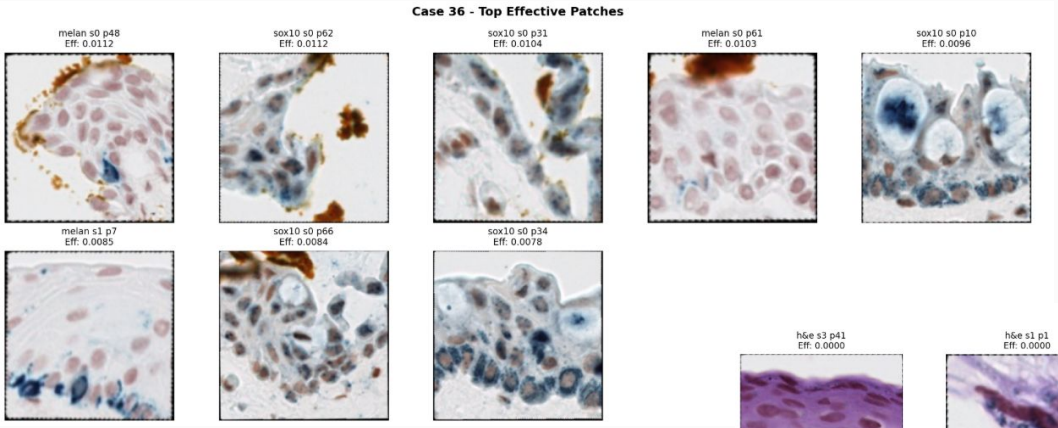


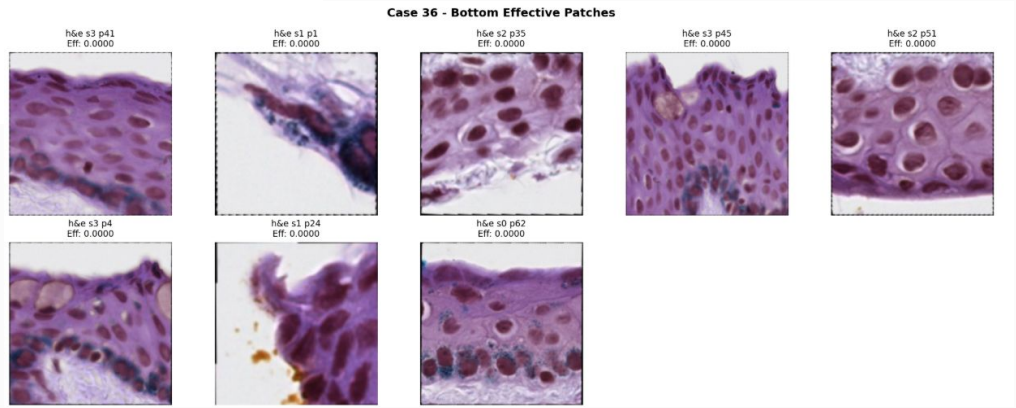
Correctly Classified Benign



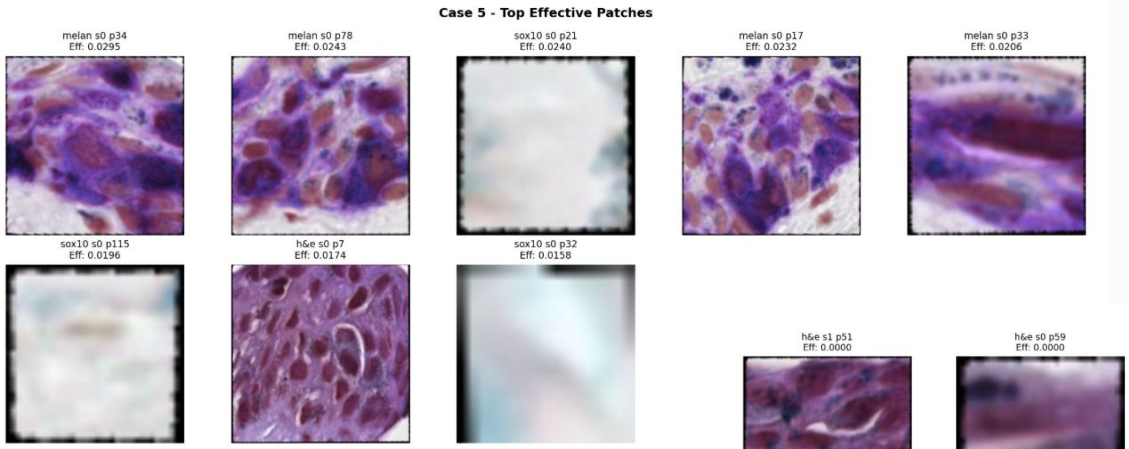


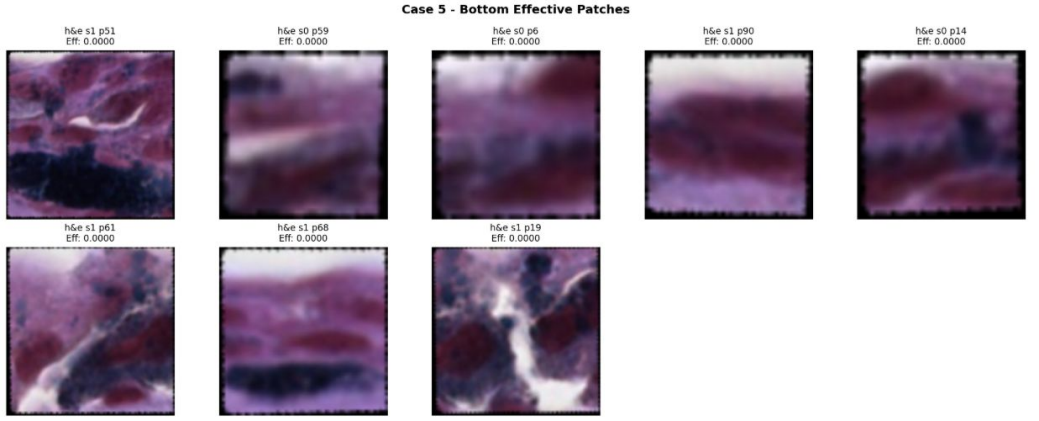
Correctly Classified Benign



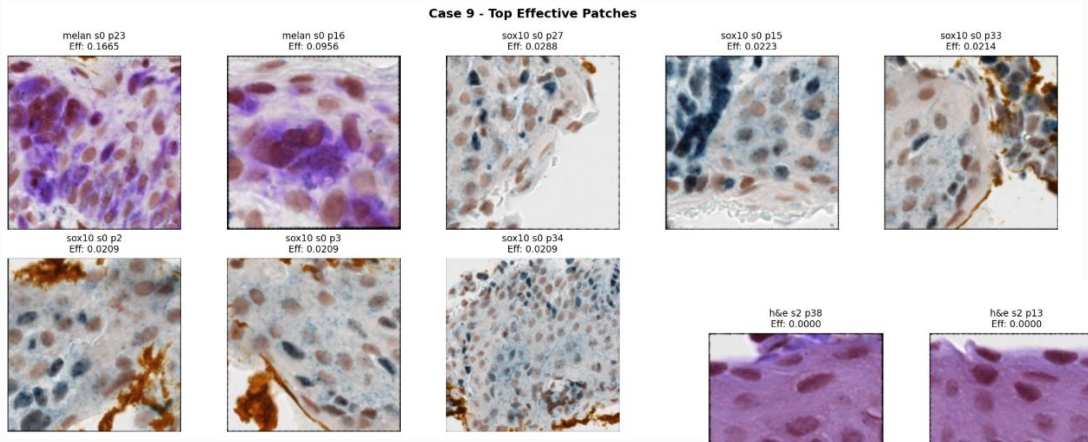


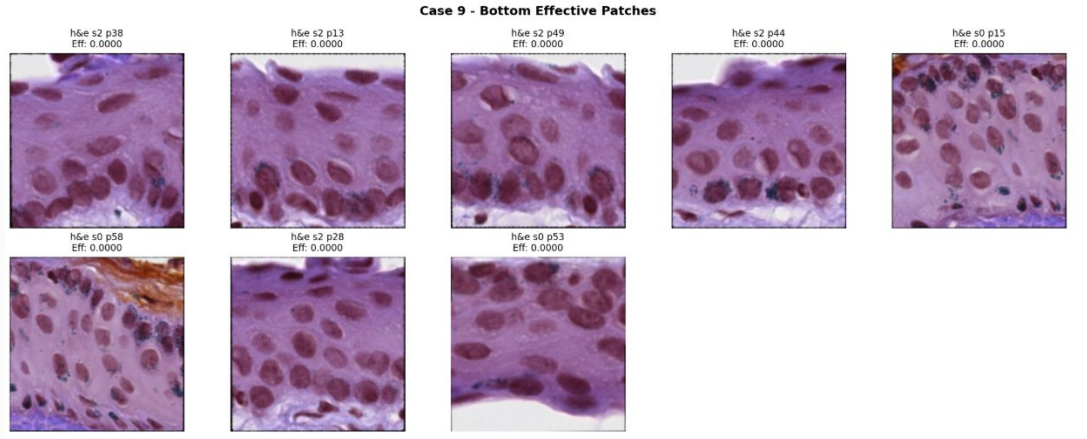
Correctly Classified High-Grade



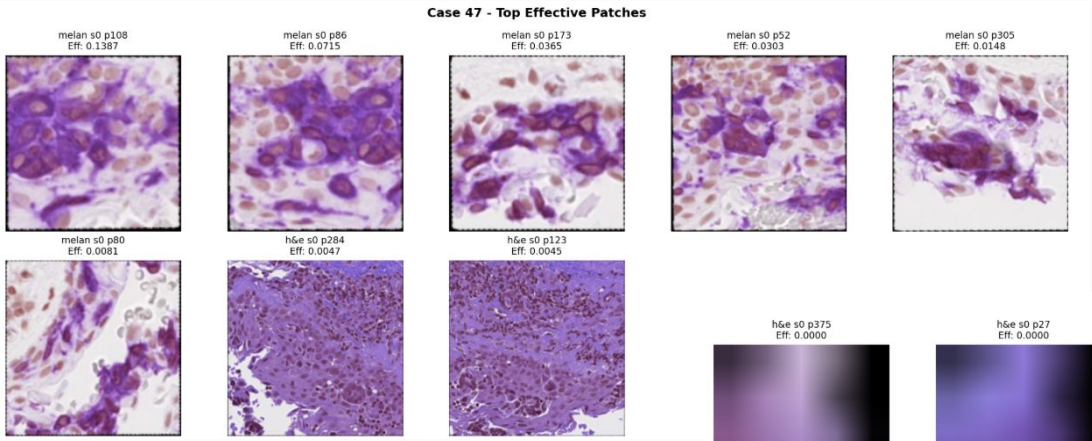


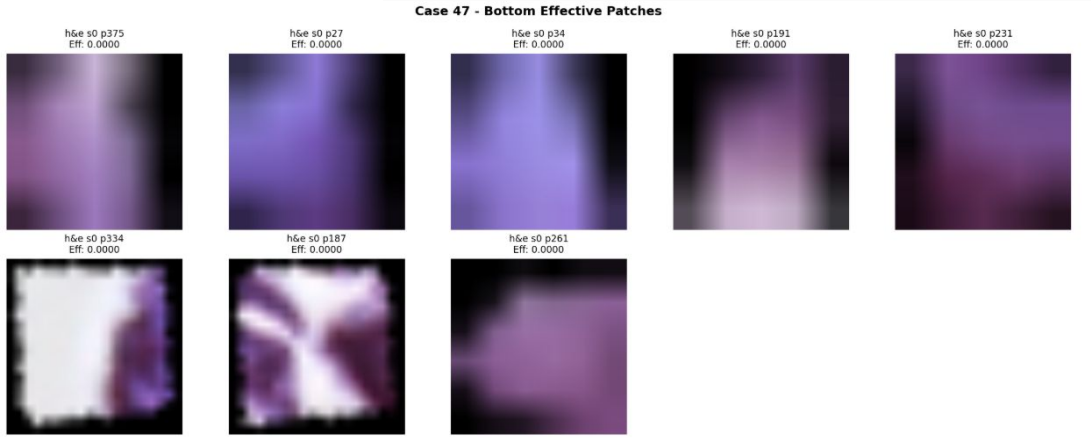
Correctly Classified High-Grade





Correctly Classified High-Grade





Analysis

- Seems like a stain dominates attention for a case
- Some cases have patches (both top and bottom) that are very small/blurry
- The patches that have “streaky” cells or empty spaces in them seem to be attended

- Increase minimum patch size
  - Possibly also limit a maximum size relative to other patches
- During patch processing filter out patches with minimal details

Splitting Benign Cases

Did it Change Model Behavior?

Case 24

Original Contains:

2 matches

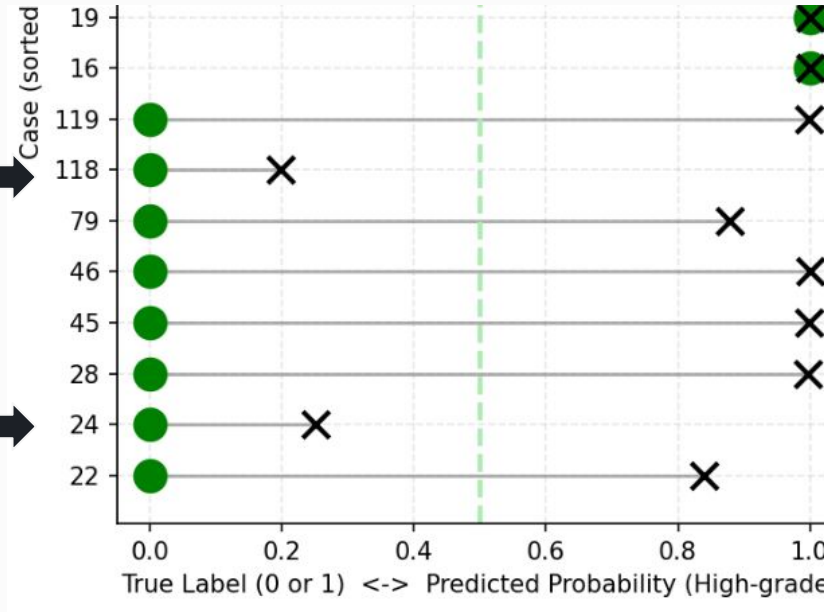
Unmatched: 8 h&e, 2 melan, 0 sox10

Case 118:

Match 1, unmatched H&E slices 1/2/3/4, unmatched melan slices 9

Case 24:

Match 2, unmatched H&E slices 5/6/7/8, unmatched melan slices 10

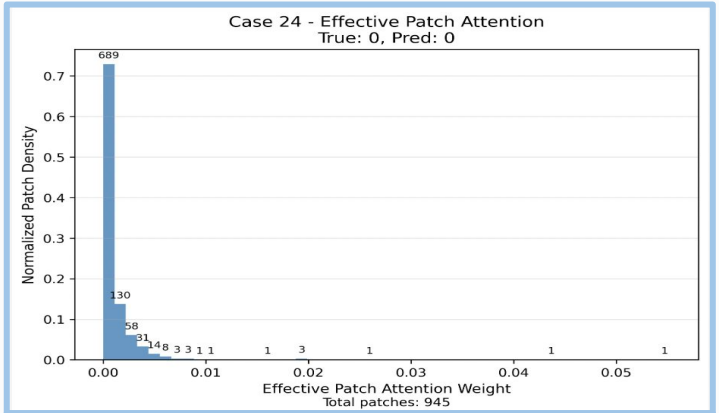


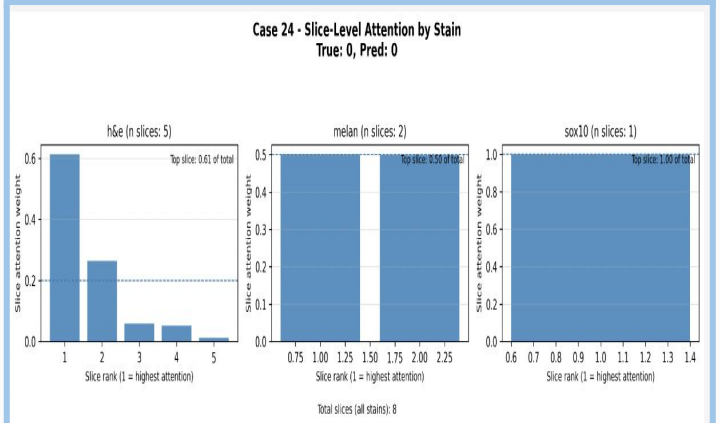
Splitting Benign Cases

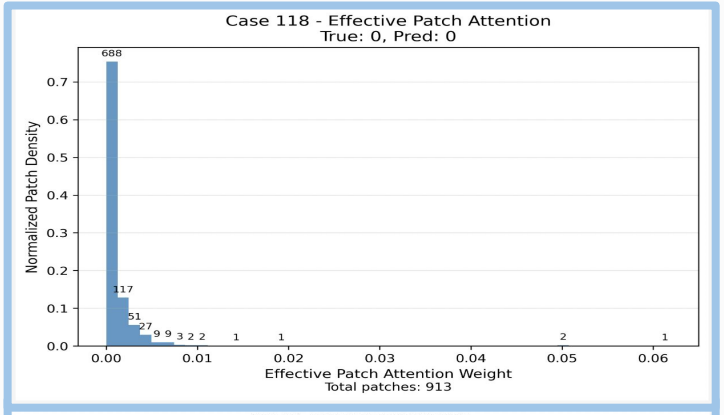
Did it Change Model Behavior?

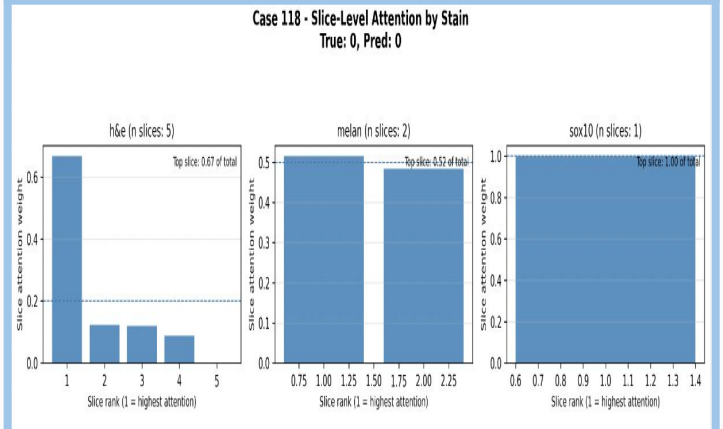
Split did not distort model behavior

Case 24









Case 118